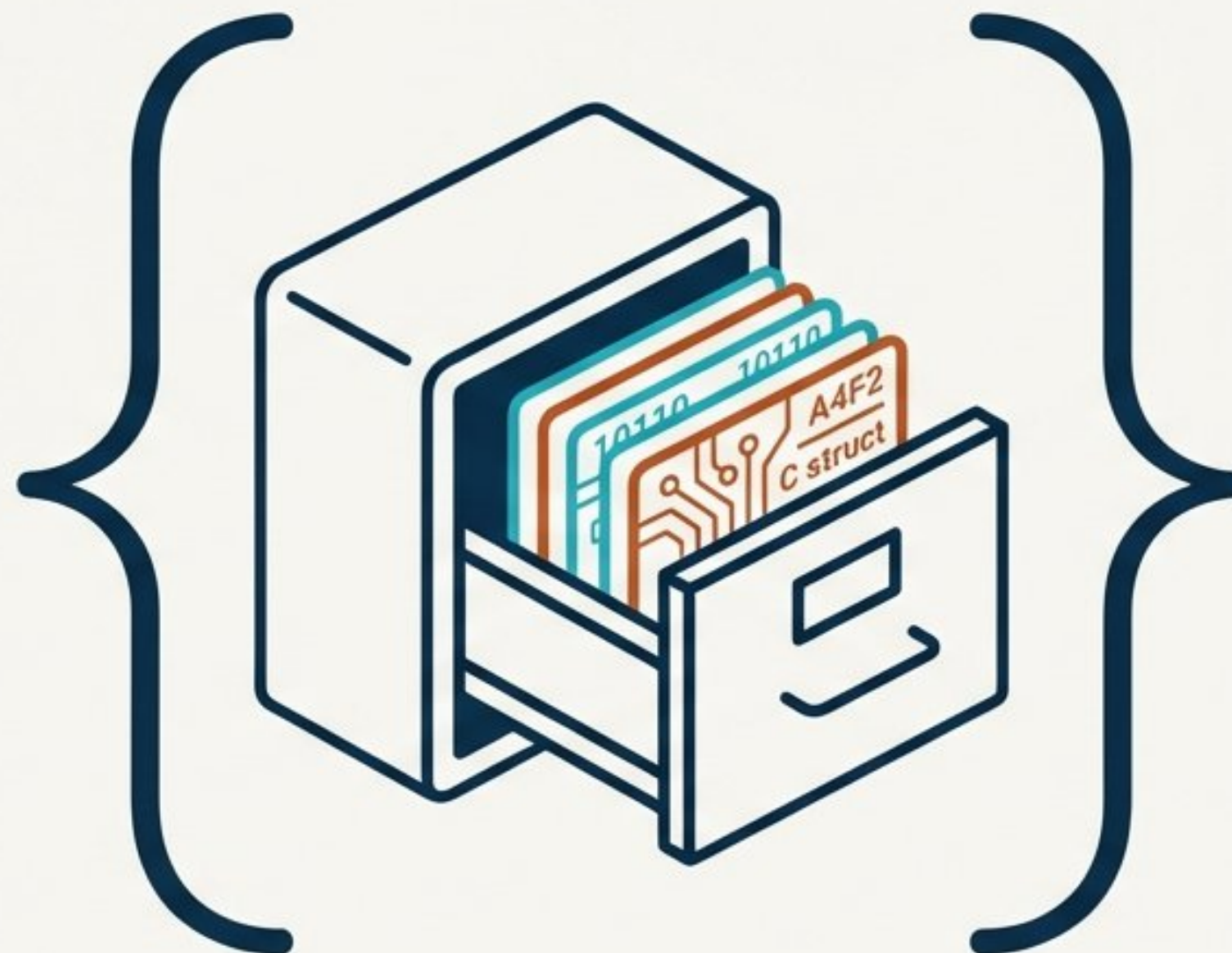




FORMAÇÃO PROFISSIONAL NÍVEL TÉCNICO – DESENVOLVIMENTO DE SISTEMAS

TEC0790 – Estruturas de Dados

Semana 5 – Estruturas de Dados Heterogêneas: struct em C

Rota de Aprendizagem da Semana



Compreender

O conceito de dados heterogêneos e sua aplicação na modelagem do mundo real.



Declarar

Tipos estruturados utilizando a palavra-chave struct na linguagem C.



Agrupar

Diferentes tipos de dados nativos (int, float, char) em uma única entidade lógica.



Manipular

Vetores de struct para resolver problemas clássicos de cadastro, registro e busca.

O Paradigma dos Dados: Vetores vs. Estruturas

Estruturas Homogêneas (Semanas 2 a 4)

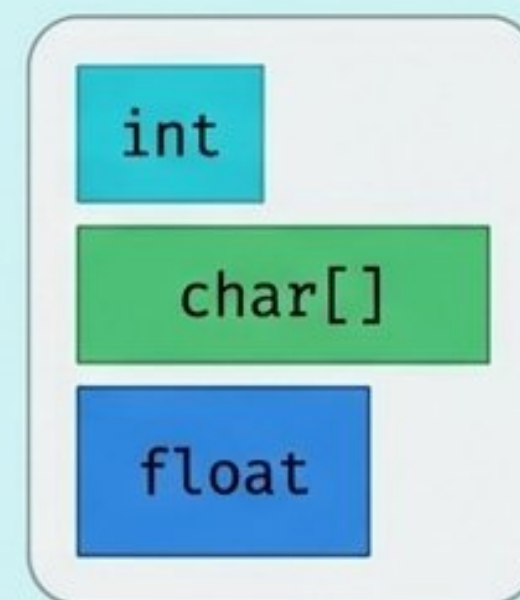
```
float notas[50];
```



Todos os elementos compartilham o mesmo tipo de dado. Ineficiente para modelar objetos complexos do mundo real.

Estruturas Heterogêneas (Semana 5)

```
struct Aluno;
```



Elementos de tipos variados unidos sob uma única identidade na memória. Modela a realidade de forma coesa.

O Fichário Digital: O Conceito de struct

A Metáfora

Uma struct (estrutura) é um novo tipo de dado composto, criado pelo programador. Ela funciona como a planta baixa de uma entidade.



Vantagem Arquitetural

Em vez de sincronizar múltiplos vetores separados, criamos uma entidade lógica. Isso agrupa campos relacionados e protege a integridade dos dados.

Sintaxe e Anatomia da **struct**

```
struct Aluno {  
    int matricula;  
    char nome[50];  
    float nota;
```

Palavra-chave obrigatória + Nome da estrutura (iniciada em maiúscula por convenção).

Variáveis aninhadas, chamadas de **campos** ou **membros**. Podem ser de qualquer tipo primitivo do C.

```
};
```

Ponto e vírgula obrigatório. O esquecimento deste caractere é o erro sintático mais comum ao declarar estruturas!

Instanciação: Do Molde à Memória

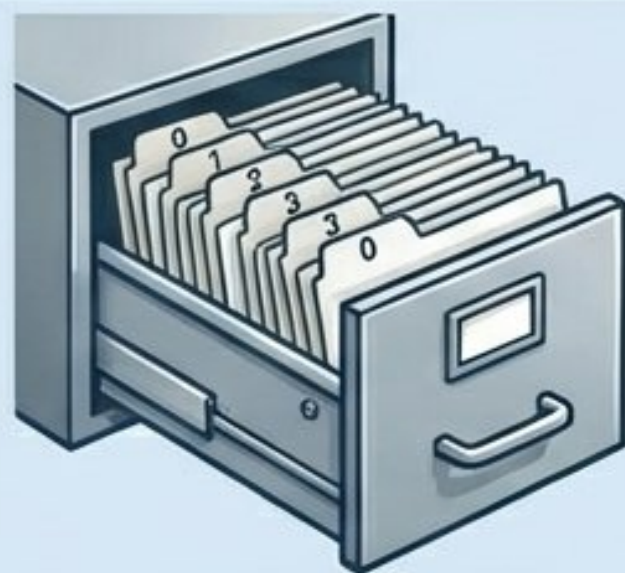
A declaração não ocupa memória, apenas **cria o molde**. Precisamos instanciar variáveis deste novo tipo.



Variável Simples

```
struct Aluno a1;
```

Cria um registro isolado de aluno na memória.



Coleções/Vetores

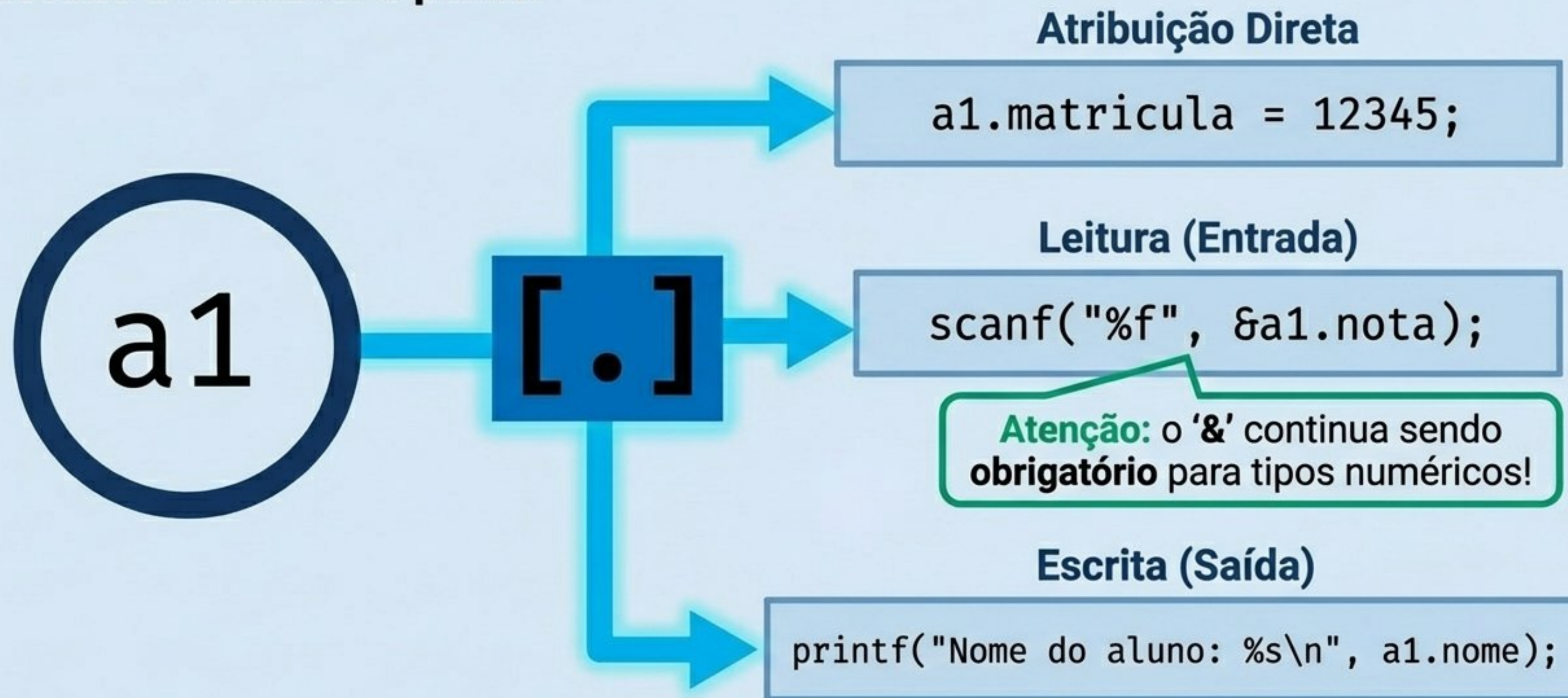
```
struct Aluno turma[50];
```

Determine o tamanho!

Cria um vetor capaz de armazenar 50 registros completos de alunos em posições contíguas.

O Operador Ponto (.) – Acessando Dados

Para ler ou gravar informações dentro de uma **struct**, usamos o operador de acesso a membro: o **ponto**.



Matriz de Registros: **Acessando Vetores de struct**

A sintaxe exige a união do índice do vetor com o operador de campo.

	.matricula	.nome	.nota
turma[0]	101	Ana	8.0
turma[1]	102	Bruno	7.5
turma[2]	103	Carlos	9.5

```
turma[2].nota = 9.5;
```

Acessa diretamente o campo nota do terceiro aluno cadastrado no vetor.

Lógica de Processamento: Percorrendo Cadastros

Uso de laços de repetição **for** é essencial para automatizar a leitura e impressão de dados em massa.

```
for (int i = 0; i < N; i++) {  
    printf("Digite a matrícula do aluno %d: ", i+1);  
    scanf("%d", &turma[i].matricula);  
}
```

O índice [i] muda a cada iteração do laço, avançando para a próxima ficha do fichário.

 **Dica:** Sempre valide o tamanho lógico (N) contra o tamanho físico máximo do vetor para evitar acesso a zonas de memória não alocadas.

Estudo de Caso 1: Relatório Acadêmico

Objetivo: Ler dados de N alunos, armazená-los e calcular a média geral da turma.

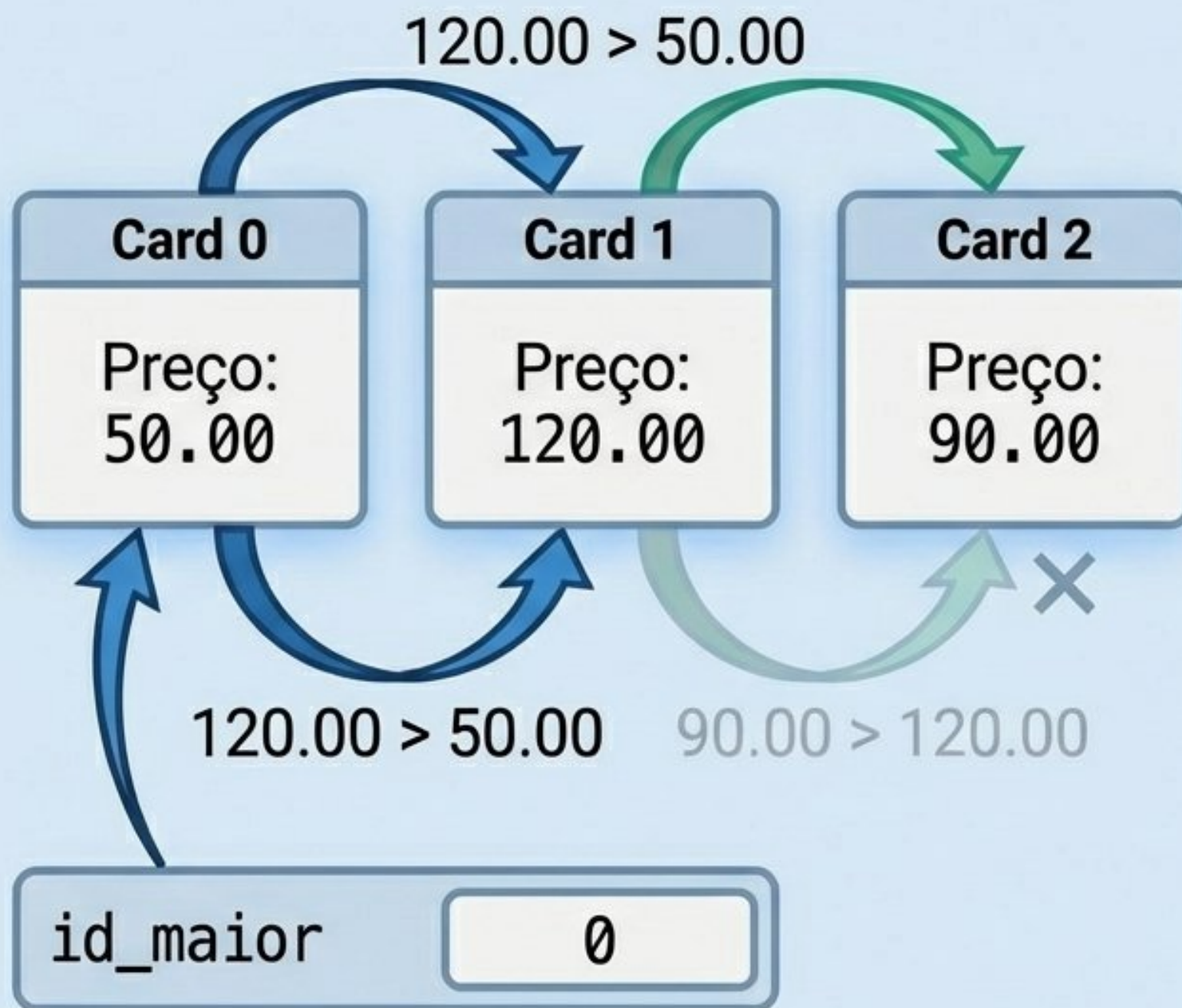
1. Declarar o vetor:
struct Aluno turma[N];
2. Usar um laço **for** para preencher os dados via teclado.
3. Usar variável acumuladora:
soma += turma[i].nota;
4. Imprimir relatório e dividir a soma por N.

```
> Lendo dados do Aluno 1...  
> Matrícula: 101  
> Nota: 8.5  
> Lendo dados do Aluno 2...  
> Matrícula: 102  
> Nota: 9.0
```

[Média da Turma: 8.75]

Estudo de Caso 2: Busca de Extremos no Estoque

Objetivo: Encontrar o item mais caro em um vetor de struct Produto.



A Lógica

- **Inicialização:**
`int id_maior = 0;`
- **Comparação:**
`if (estoque[i].preco > estoque[id_maior].preco)`
- **Atualização:**
`id_maior = i;`
- **Resultado:**
`printf("Mais caro: %s",
estoque[id_maior].descricao);`

Prevenção de Erros e Boas Práticas



O Esquecimento Fatal

Não esqueça o ';' após a chave de fechamento da declaração da struct. O compilador gerará erros massivos.



Leitura de Strings

Cuidado ao usar **scanf** para textos com espaços (nomes completos). Prefira funções seguras para preencher `char[]` dentro de structs.



Nomeação Semântica

Usar **p.preco** é infinitamente superior a **p.x**. Nomes de entidades devem iniciar em Maiúscula (Produto, Aluno).



Identação Rigorosa

Mantenha os campos alinhados visualmente dentro da struct para facilitar a manutenção futura.

Missão Prática de Laboratório

Construção de um sistema de cadastro de pequeno porte.

LABORATÓRIO



1. **DECLARE** uma struct Aluno contendo os campos matricula (int), nome (string), e nota (float).
2. **CRIE** um programa principal que declare um vetor dessa struct com capacidade para 3 alunos.
3. **PROCESSE** a entrada lendo os dados de todos os alunos via teclado.
4. **EXIBA** os dados percorrendo o vetor com um laço for para imprimir, em formato de tabela limpa, todos os alunos cadastrados.

Critérios de Êxito: Correção sintática absoluta e uso impecável do operador ponto (.).

CÓDIGO

```
#include <iostream>
#include <iomanip> // necessário para setprecision
#include <string> // necessário para usar std::string
using namespace std;
```

```
struct Aluno {
    int matricula;
    string nome; // agora usamos string em vez de char[]
    float nota;
};
```

```
int main() {
    const int n = 3; // exemplo com 3 alunos
    Aluno turma[n];
    float soma = 0, media;
    int indiceMaiorNota = 0;
```

```
cout << "--- SEMANA 5: CADASTRO DE ALUNOS (STRUCT) ---\n\n";
```

```
// Leitura dos dados
```

```
for (int i = 0; i < n; i++) {
    cout << "Dados do Aluno " << i + 1 << ":\n";
```

```
    cout << "Matricula: ";
    cin >> turma[i].matricula;
    cin.ignore(); // descarta o '\n' antes de ler o nome
```

```
    cout << "Nome: ";
    getline(cin, turma[i].nome); // leitura direta em string
```

```
    cout << "Nota: ";
    cin >> turma[i].nota;
```

```
    cout << "-----\n";
}
```

1. Estrutura do Programa

O programa começa incluindo bibliotecas:

- #include <iostream> → entrada e saída padrão.
- #include <iomanip> → formatação de números (casas decimais).
- #include <string> → uso da classe std::string.

2. Definição da Struct

struct agrupa dados diferentes em um único tipo.

Cada aluno tem:

- **matricula** (inteiro),
- **nome** (texto),
- **nota** (número decimal).

3. Declaração do Vetor

- Cria um vetor de 3 elementos do tipo Aluno.
- Permite armazenar vários registros em uma coleção.

4. Entrada de Dados

- Laço for percorre cada posição do vetor.
- cin >> lê matrícula e nota.
- getline(cin, ...) lê o nome completo (com espaços).
- cin.ignore() descarta o \n que sobra da leitura anterior.

5. Processamento

- Calcula a **soma das notas**.
- Determina o **índice do aluno com maior nota**.
- Calcula a **média da turma**.

```
// Processamento: soma e maior nota
```

```
for (int i = 0; i < n; i++) {
    soma += turma[i].nota;
    if (turma[i].nota > turma[indiceMaiorNota].nota) {
        indiceMaiorNota = i;
    }
}
```

```
media = soma / n;
```

```
// Relatório
```

```
cout << "\n--- RELATORIO DA TURMA ---\n";
for (int i = 0; i < n; i++) {
    cout << "Mat: " << turma[i].matricula
        << "| Nome: " << turma[i].nome
        << "| Nota: " << fixed << setprecision(2) << turma[i].nota <<
        "\n";
}
```

```
// Saída com duas casas decimais
```

```
cout << "\nMedia da Turma: " << fixed << setprecision(2) << media
<< "\n";
cout << "Aluno com maior nota: " << turma[indiceMaiorNota].nome
<< "(Nota: " << fixed << setprecision(1) <<
turma[indiceMaiorNota].nota << ")\n";

return 0;
}
```

4. Saída Formatada

- fixed → usa notação decimal fixa.
- setprecision(2) → mostra duas casas decimais.
- Exibe relatório com todos os alunos e destaca:
 - Média da turma.
 - Aluno com maior nota.



Atividade Verificadora

Cenário: Sistema de Gestão de Inventário.

O Requisito:

Desenvolver, em linguagem C, um programa completo que obedeça aos seguintes critérios:

- Defina uma **struct Produto** contendo `codigo (int)`, `descricao (char[])`, e `preco (float)`.
- Leia e armazene os dados de exatamente 5 produtos em um vetor.
- **Processe** o vetor para localizar o produto de maior valor financeiro.
- Imprima exclusivamente o relatório completo (código, descrição e preço) deste produto mais caro.

Entrega: Codifique na IDE, compile, teste com dados variados e submeta conforme instrução do docente na plataforma virtual.